ME 235 Lab 3

Rupali Batta and Sophie Steven

April 2026

1 Introduction

2 Part 1: Vision in the Loop

2.1 Pre-lab Questions

2.1.1 (a)

The PnP problem is Perspective-n-Point (PnP) and it is the problem of estimating the pose of a calibrated camera relative to a rigid object, given:

- the camera intrinsics (focal lengths, principal point) and distortion,
- a set of 3D points expressed in an object- or marker-fixed frame, and
- their matching 2D projections in the image.

The unknown is the rotation and translation that relate the two coordinate systems.

After `detectMarkers` finds each tag's four corners in pixel coordinates, those corners are paired with the real measured dimensions of the square marker in its own frame. Then `solvePnP` solves for the rigid transform from the marker frame to the camera frame.

2.1.2 (b)

For planar images like ArUco, four point pairs (the four corners of one square) are enough for a stable pose with a calibrated camera since they are non-collinear and fix the planar pose up to the usual degeneracies avoided in normal viewing. Fewer than four coplanar points are not enough for full pose.

There are many different ArUco markers not because PnP needs more 3D points, but to distinguish markers between itself and make it more consistent at detecting them Dictionaries are designed with Hamming distance between patterns so bits can be read robustly given the camera noise.

2.2 Lab Report Questions

2.2.1 (a)

The IDs and transforms detected in the in-lab test image are:

Tag ID 3

$$T_{\text{base,tag}} = \begin{bmatrix} 0.817674 & -0.432613 & 0.379808 & -0.056088 \\ 0.382895 & 0.901357 & 0.202353 & -0.432956 \\ -0.429884 & -0.020032 & 0.902662 & -0.011596 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Tag ID 4

$$T_{\text{base,tag}} = \begin{bmatrix} 0.479888 & 0.876466 & 0.038921 & -0.124950 \\ -0.873775 & 0.481463 & -0.068635 & -0.381085 \\ -0.078896 & -0.001071 & 0.996882 & -0.074780 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Tag ID 2

$$T_{\text{base,tag}} = \begin{bmatrix} 0.448041 & 0.877030 & 0.173432 & -0.190927 \\ -0.853545 & 0.477339 & -0.208826 & -0.344498 \\ -0.265932 & -0.054469 & 0.962452 & -0.123107 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2.2 (b)

The most uncertain axis in the tag pose estimation is the **z-axis (depth)**.

This is because depth must be inferred from perspective projection rather than directly measured using the ARuCO tags, the way the x and y axes can. Small errors in corner detection lead to large uncertainty in estimated distance.

To improve state estimation, we could

- Use known geometric constraints (ie. blocks lie on a table and the table is at 0, and we know the heights of each block).
- Average estimates over multiple frames.
- Fuse with robot forward kinematics for consistent camera pose.
- Use multiple viewpoints or additional sensors such as depth cameras.

In the Tower of Hanoi setup, we improve accuracy by hardcoding known block heights.

3 Part 2: Parallel Jaw Grasping

3.1 Pre-lab Questions

3.1.1 (a)

You typically do not move directly from far away into the final grasp pose in a single motion. Instead, you first move to a pre-grasp (approach) pose, where:

- the gripper jaws are open to the correct width, and
- the TCP is offset slightly along the approach direction.

This offset prevents colliding with the object and gives more of a tolerance.

The appropriate distance depends on the TCP definition, finger geometry and stroke, object width and thickness, object shape (simple prism vs. irregular geometry), and material properties (fragile objects require more cautious approaches).

3.1.2 (b)

Work in a consistent reference frame (e.g., robot base or world). If the object pose is given in the camera frame:

$$T_{\text{world, object}} = T_{\text{world, camera}} \cdot T_{\text{camera, object}}$$

To compute a grasp:

- Select a grasp frame G on the object.
- Define $T_{\text{object, grasp}}$.
- Compose transforms to obtain $T_{\text{world, TCP}}$.
- Solve IK and execute.

3.2 Lab Report Questions

3.2.1 (a)

The computed grasp transforms are:

Tag ID 3

$$T_{\text{bg}} = \begin{bmatrix} 0.905625 & 0.424080 & 0 & -0.040686 \\ 0.424080 & -0.905625 & 0 & -0.381586 \\ 0 & 0 & -1 & 0.370408 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Tag ID 4

$$T_{\text{bg}} = \begin{bmatrix} 0.481389 & -0.876507 & 0 & -0.070696 \\ -0.876507 & -0.481389 & 0 & -0.396777 \\ 0 & 0 & -1 & 0.322022 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Tag ID 2

$$T_{\text{bg}} = \begin{bmatrix} 0.464776 & -0.885428 & 0 & -0.137924 \\ -0.885428 & -0.464776 & 0 & -0.359546 \\ 0 & 0 & -1 & 0.264077 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.2.2 (b)

If the objects are not perfect rectangular prisms but their geometry is known, one approach is to compute grasps based on surface geometry by identifying opposing contact surfaces and defining a grasp frame aligned with surface normals. This is similar to a paper discussed in lecture using deep learning where the authors found a suitable "plane" that was normal to two irregular faces of an object and gripped onto that plane.

4 Part 3: Tower of Hanoi

No pre-lab questions for this section.

4.1 Lab Report Questions

We did not succeed in fully completing the Tower of Hanoi puzzle, but the robot was able to pick up the blocks and drop them down into roughly the correct place. To make the dropping off more accurate, we would just have to finetune the hardcoded z that we set for where to drop them, so that the robot would gently place it on the table instead of dropping from six inches up. Also the y was slightly off in picking up the boxes, which could also be solved by finetuning the approach offset. Finally, the actual solving of the Tower of Hanoi was incorrect (the robot went for the wrong block after putting down the tiniest top one). We had recursive logic function for solving it but didn't realize we could have hardcoded the order of which blocks to put where. So doing that would definitely fix that issue.